

# Relazione progetto

Nome: Mattia Salinitri

## Descrizione del torneo scelto

Ho scelto di realizzare un gestionale per un torneo di calcio. L'applicazione permette di gestire le squadre partecipanti, i giocatori di ogni squadra e le partite del torneo. Ogni partita viene giocata tra due squadre (una "in casa" e una "ospite") e alla fine si registra il risultato come numero di gol segnati da ciascuna. In base ai risultati inseriti, il programma calcola in automatico la classifica del torneo, assegnando 3 punti a chi vince, 1 punto a testa in caso di pareggio e 0 punti a chi perde, come si fa normalmente nel calcio.

## Struttura del database

Il database è stato realizzato con SQL Server (LocalDB), quello incluso in Visual Studio, quindi non serve installare un server esterno. È composto da 3 tabelle collegate tra loro con chiavi esterne:

| Tabella   | Chiave primaria | Colonne principali                                                                 |
|-----------|-----------------|------------------------------------------------------------------------------------|
| Squadre   | IdSquadra       | Nome, Città, AnnoFondazione                                                        |
| Giocatori | IdGiocatore     | Nome, Cognome, Ruolo, NumeroMaglia, IdSquadra (FK)                                 |
| Partite   | IdPartita       | IdSquadraCasa (FK), IdSquadraOspite (FK), DataPartita, GolCasa, GolOspite, Giocata |

La tabella Giocatori ha una chiave esterna verso Squadre (ogni giocatore appartiene a una squadra), mentre la tabella Partite ha due chiavi esterne verso Squadre, perché una partita coinvolge sempre due squadre diverse (casa e ospite). Lo script SQL completo per creare il database da zero si trova nel file script.sql caricato nella repository.

## Architettura del progetto (MVC)

Il progetto segue il pattern MVC, quindi il codice è diviso in cartelle separate a seconda del compito che svolgono:

- Models: le classi che rappresentano i dati
- Data: la classe ClsSQLServer che si occupa di aprire la connessione al database ed eseguire le query e DbConfig che contiene la stringa di connessione.
- Controllers: una classe per ogni entità che contiene tutte le query SQL e la logica per inserire, modificare, eliminare e leggere i dati.
- Views: i Form chiamano i metodi dei Controller e non contengono mai query SQL scritte direttamente.

## Descrizione della funzionalità AI

Tra le funzionalità di intelligenza artificiale proposte nella consegna ho scelto "previsione del possibile vincitore di una partita in base ai risultati precedenti", implementata nella classe PrevisioneController.

Non è un modello di machine learning vero e proprio (non ho usato librerie di AI né modelli addestrati), ma un algoritmo che calcola in modo automatico una previsione partendo dai dati reali già presenti nel database. Funziona così:

- Per ogni squadra si guardano tutte le partite già giocate (quelle con Giocata = 1).
- Per ogni partita si calcolano i punti ottenuti (3 se vinta, 1 se pareggiata, 0 se persa) e la differenza reti (gol fatti meno gol subiti).

- Si fa la media dei punti e la media della differenza reti su tutte le partite giocate, ottenendo un "indice di forza" della squadra:  $\text{forza} = \text{punti medi} + (\text{differenza reti media} \times 0,2)$ .
- Alla squadra che gioca in casa viene aggiunto un piccolo bonus (+0,3), per simulare il classico vantaggio del fattore campo.
- Le due forze (casa e ospite) vengono trasformate in percentuali di vittoria/pareggio/sconfitta che sommano al 100%, mostrate all'utente insieme a un breve commento generato in automatico.

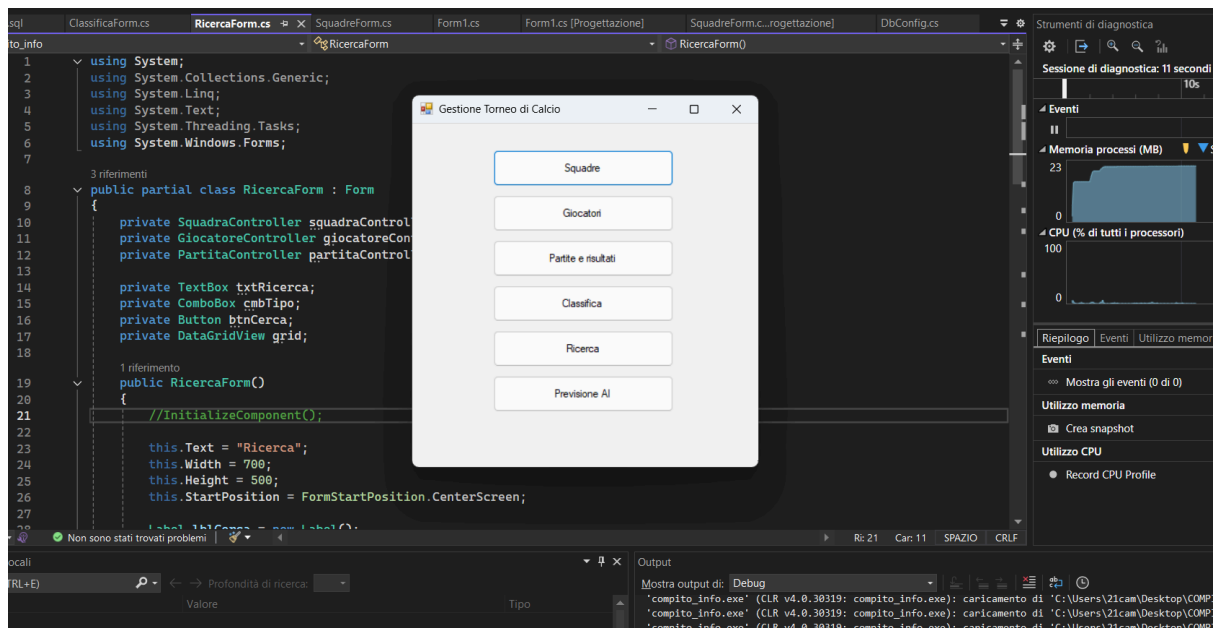
Se una squadra non ha ancora giocato nessuna partita, le viene assegnata una forza "neutra" (1,0), così la previsione non va in errore e resta comunque sensata.

## Dati utilizzati dalla funzionalità AI

I dati usati dall'algoritmo sono esclusivamente quelli già presenti nel database del torneo: le partite della tabella Partite che risultano già giocate (Giocata = 1), con i relativi gol segnati e subiti dalle due squadre. Non vengono usati dati esterni o presi da internet: la previsione si basa solo sullo storico che l'utente stesso inserisce usando l'applicazione.

## Screenshot dell'applicazione

Schermata: Menu principale (Form1)



Schermata: Gestione Squadre

Gestione Squadre

Nome:

Città:

Anno fondazione:

|   | IdSquadra | Nome      | Città  | AnnoFondazione |
|---|-----------|-----------|--------|----------------|
| ▶ | 4         | Inter     | Milano | 1908           |
|   | 2         | Juventus  | Torino | 1897           |
|   | 3         | Milan     | Milano | 1899           |
|   | 1         | Torino FC | Torino | 1906           |

Schermata: Gestione Giocatori

Gestione Giocatori

Nome:  Squadra:

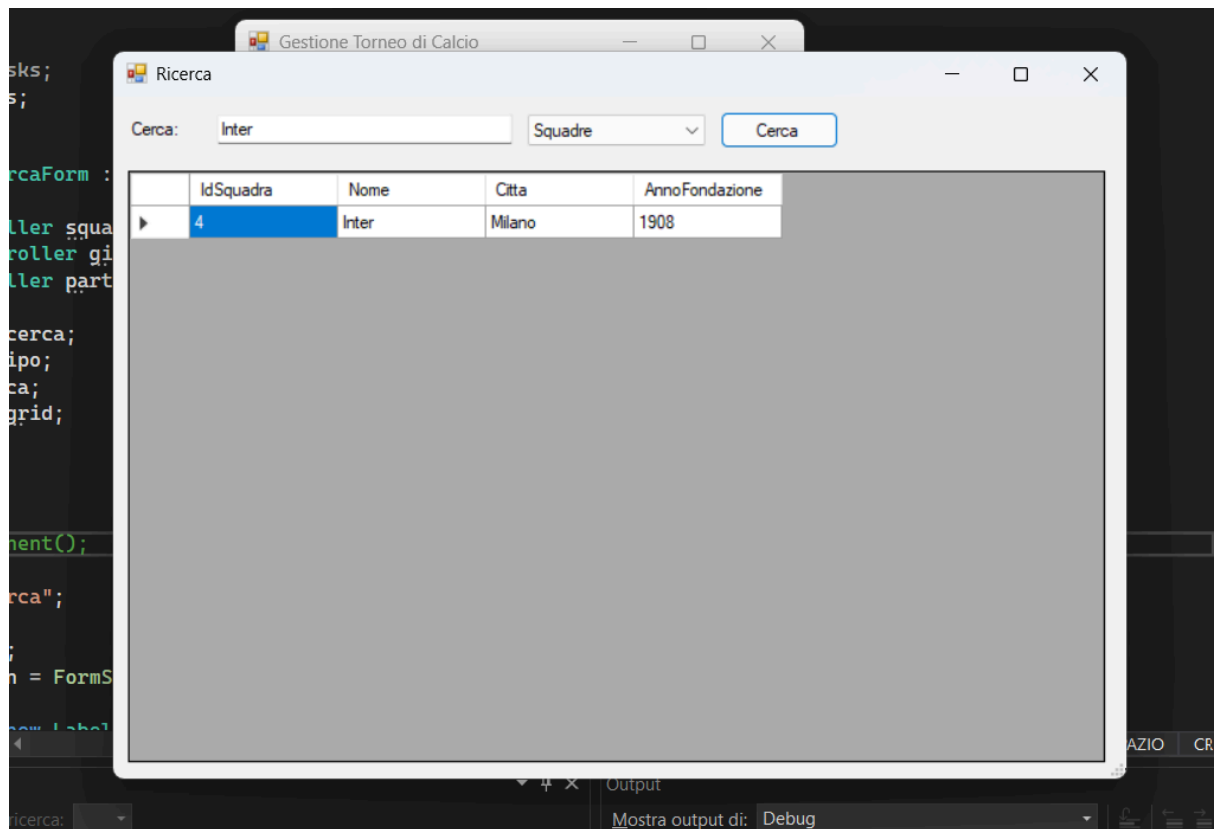
Cognome:

Ruolo:

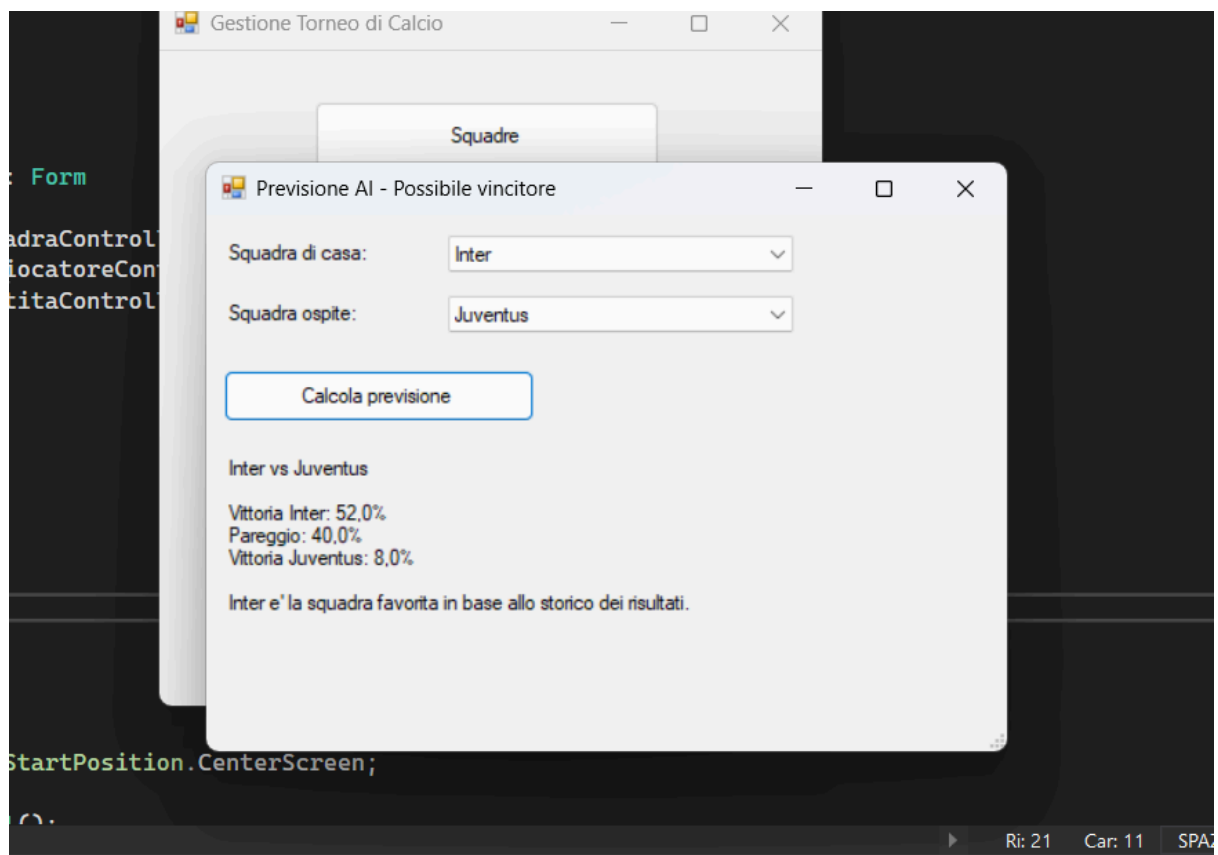
Numero maglia:

|   | Nome  | Cognome | Ruolo      | NumeroMaglia | IdSquadra | NomeSquadra |
|---|-------|---------|------------|--------------|-----------|-------------|
| ▶ | Paolo | Verdi   | Difensore  | 4            | 2         | Juventus    |
|   | Luca  | Bianchi | Portiere   | 1            | 1         | Torino FC   |
|   | Marco | Rossi   | Attaccante | 9            | 1         | Torino FC   |

Schermata: Ricerca



Schermata: Previsione AI



## Conclusioni

Realizzando questo progetto ho potuto mettere in pratica quello che abbiamo fatto durante l'anno su C#, il pattern MVC e le basi di dati relazionali, unendo tutto in un'unica applicazione funzionante. La parte più complessa è stata sicuramente il calcolo della classifica, perché ogni squadra gioca sia in casa che in trasferta e quindi le statistiche vanno sommate da due query diverse. Anche capire come collegare correttamente i Form al database tramite i Controller, senza scrivere query direttamente nell'interfaccia grafica, mi ha aiutato a capire meglio perché si usa il pattern MVC nei progetti reali.